CLOUD ENGINEERING ASSIGNMENT

Ansible Risk Review

Agentic Loop + Claude Code Change Analysis Guide

A complete walkthrough of building a Bash-based risk-review workflow, using Claude Code to analyze Ansible dry-run evidence, classifying risky infrastructure changes before a human approves them.

TOOL	Ansible (--check --diff mode)
AGENT	Claude Code (analysis only — no file edits)
WORKFLOW	Gather → Analyze → Human Approval
PATTERN	Agentic Loop | Human in the Loop

Shared for educational purposes on LinkedIn

Follow along, learn, build, and share with someone who needs it.

8 TASKS
covered end-to-end

Contents

How This Workflow Works — The Agentic Loop
Task 1 — Verify Ansible Can Reach the VM
Task 2 — Project Safety Rules (CLAUDE.md)
Task 3 — The Three Phases: Gather, Analyze, Human Approval
Task 4 — The Bash Risk-Review Script
Task 5 — Baseline Run Results
Task 6 — The Claude Code Skill (Permissions + Division of Work)
Task 7 — Injecting and Detecting a Risky Change
Task 8 — Human Approval and Final Apply
All Assignment Q&A; Answers
Next-Time Quick Checklist

How This Workflow Works — The Agentic Loop

This assignment builds a change-safety workflow around Ansible. Instead of running the real playbook and hoping for the best, we run a dry run first, let a Bash script classify the risk, and let Claude Code explain the findings in plain language. A human then decides whether to proceed.

PHASE	WHO DOES IT	WHAT HAPPENS
Gather	Bash script	Runs ansible-playbook --check --diff, captures output, detects changed task
Analyze	Claude Code	Reads the Bash report, explains what is risky and why in plain language
Human Approval	You	Reviews Claude Code's analysis and decides whether to run the real playbo
Apply (optional)	You (manually)	Runs the real ansible-playbook only after approving the analysis

Note: *Claude Code only reads and explains. It never runs the real playbook, never edits files, and never applies changes. Every real change requires a human decision.*

Exit codes the Bash script returns:

EXIT CODE	MEANING	ACTION REQUIRED
0	Healthy — no changes detected	Safe to proceed
1	Warning — changes present	Review recommended before applying
2	Risky — high-risk change detected	Human must approve before applying

TASK 1 Verify Ansible Can Reach the VM

```
ansible web -i inventory.ini -m ping

# Expected result:
epicbook | SUCCESS
"ping": "pong"
```

Q: What proves Ansible can reach the EpicBook VM?

The pong response to the ping module proves the Ansible controller successfully communicated with the VM over SSH.

Q: Why confirm playbook syntax before building the risk-review script?

The risk-review script depends on the playbook being valid. If the playbook has syntax errors, the workflow cannot produce meaningful change information. Confirming syntax first ensures the underlying playbook can be parsed.

TASK 2 Project Safety Rules — CLAUDE.md

Before using Claude Code on any infrastructure project, you define explicit safety rules in a CLAUDE.md file at the project root. This tells Claude Code exactly what it is and is not allowed to do.

```
# CLAUDE.md — project safety rules

This project is an Ansible risk-review workflow.

Allowed:
- Read files
- Run Bash commands (the risk-review script)
- Analyze and explain Ansible dry-run output

Not allowed:
- Create or edit any files
- Run the real Ansible playbook
- Never apply, converge, or fix the playbook automatically
```

Q: Why should Claude Code have project-specific safety rules?

Claude Code needs to understand the purpose and boundaries of the project. In this assignment it was being used to analyze changes, not apply them. The rules made that distinction explicit.

Q: Why should the human run the real Ansible playbook manually?

The real playbook can change a server. The human should review the evidence and make the final decision because configuration changes can affect availability, security, permissions or data.

Q: Which rule prevents Claude Code from applying changes automatically?

The rule: Never apply, converge, or fix the playbook automatically.

TASK 3 The Three Phases: Gather, Analyze, Human Approval

GATHER	Bash script runs ansible-playbook --check --diff and stores the raw output in a report file.
ANALYZE	Claude Code reads the report and explains which tasks would change, their risk category, and likely impact.

APPROVE	You review Claude Code's analysis and decide whether to run the real playbook manually.

Q: Which part is the Gather phase?

The Gather phase is ansible-playbook --check --diff. The Bash script runs this, captures the output and stores the raw Ansible results.

Q: Which part is the Analyze phase?

The Analyze phase is Claude Code reading the generated report and explaining which changes are risky, what category they fall into, and what their likely impact is.

Q: How was it verified that Claude Code did not create or edit files?

The workflow was designed as read-only analysis. The CLAUDE.md safety rules explicitly prohibited editing, and the skill itself was restricted to Bash, Read and Grep.

TASK 4 The Bash Risk-Review Script

The Bash script does all the deterministic work so Claude Code only has to read and explain the results.

What the script does:

1	Runs ansible-playbook --check --diff against the inventory
2	Captures the full Ansible output
3	Calls extract_changed_tasks() to find tasks Ansible would change
4	Classifies each changed task against the four high-risk categories
5	Generates a structured report file
6	Returns exit code 0 (healthy), 1 (warning) or 2 (risky)

The four high-risk categories:

RISK CATEGORY	EXAMPLES
Package / File Removal	rm, delete, absent state on a file or package
Permission Changes	chmod, chown, user permission modifications
Service Disruption	stop, disable, restart of a running service
Security Configuration	firewall rules, SSH config, auth changes

Q: What is stored in the changed_tasks array?

Task names that Ansible reports as changes during the dry run.

Q: Which function finds changed tasks?

The extract_changed_tasks function.

Q: Why does the script use --check --diff?

--check previews the playbook without applying changes. --diff shows exactly what would be different in supported modules.

Q: Why does the script use different exit codes?

Different codes let both humans and automation understand the result: 0 = healthy, 1 = warning (review recommended), 2 = risky (must approve).

TASK 5 Baseline Run Results

The first run with no intentional risky changes:

```
PLAY RECAP
epicbook : ok=XX changed=XX unreachable=0 failed=0

Status: WARN - changes present, review recommended
Exit Code: 1

Changed tasks detected:
common : Update apt cache
epicbook: Install EpicBook dependencies

Risky tasks detected: none
```

Q: What was the overall baseline status?

WARN — changes present, review recommended. The dry run completed with unreachable=0 and failed=0.

Q: Did any tasks report changed?

Yes. Two tasks were detected: common: Update apt cache and epicbook: Install EpicBook dependencies.

Q: Were any changed tasks flagged as risky?

No. Neither of the baseline changed tasks matched the four high-risk categories.

Q: What does exit code 1 mean here?

Changes were detected and review is recommended, but none of the specifically defined risky categories were triggered.

TASK 6 The Claude Code Skill — Permissions and Division of Work

A Claude Code skill defines exactly what the agent is allowed to do. This skill deliberately keeps Claude Code in a read-only analysis role.

COMPONENT	WHAT IT HANDLES
Bash (allowed)	Runs the risk-review script, captures Ansible output, detects and classifies changes, generates report
Read (allowed)	Inspects the generated report files
Grep (allowed)	Searches output for relevant evidence
File editing (NOT allowed)	Prevented — Claude Code must not modify infrastructure files while analyzing them
Real playbook (NOT allowed)	Never run by Claude Code — human only

Q: Why does the skill allow Bash, Read and Grep but not editing?

The skill is designed to review changes, not make them. Preventing editing reduces the risk of Claude Code modifying the environment it is supposed to be analyzing.

Q: Why is this better than just asking Claude Code if the playbook is safe?

Claude Code is working from actual Ansible dry-run evidence, not guessing. The Bash script provides concrete information about the current server state and exactly what would change.

TASK 7 Injecting and Detecting a Risky Change

A test task was added to the playbook to verify the risk-detection logic works.

```
# Test task added to common role
- name: Remove temporary EpicBook risk test file
file:
path: /tmp/epicbook-risk-test.txt
state: absent
```

What the Bash report returned:

```
Status: RISKY - high-risk change detected
Exit Code: 2

Risky tasks:
risky (removal): common : Remove temporary EpicBook risk test file

Risk category: Package / File Removal
```

Important: A removal task always requires human review before the real playbook is applied. The agent flags it — the human decides.

Q: Which risk category did the added task fall into?

Package / File Removal. The report identified: risky (removal): common: Remove temporary EpicBook risk test file

Q: What evidence proves the task would change something?

The Ansible dry run reported: would change: common: Remove temporary EpicBook risk test file. The Bash report then classified it as a risky removal.

Q: Did Claude Code apply the playbook?

No. Claude Code only analyzed the report. The real playbook was later run manually by the student.

Q: Why is it important that Claude Code only analyzed the risk?

Automatically applying a destructive change removes the human approval step. Keeping Claude Code in an analysis role means the human reviews the evidence before infrastructure is changed.

Q: Which phase of the Agentic Loop does the Bash report represent?

Primarily the Gather phase — it collects evidence from the Ansible dry run. It also performs mechanical classification, which supports the Analyze phase.

TASK 8 Human Approval and Final Apply

After reviewing the risk report and Claude Code's analysis, the human makes the final decision and runs the real playbook.

```
# Human manually runs the real playbook after approving the analysis
ansible-playbook -i inventory.ini site.yml

# Verify the VM is still reachable after the change
ansible web -i inventory.ini -m ping

# Expected:
epicbook | SUCCESS
"ping": "pong"
```

Q: What command was used to apply the change for real?

ansible-playbook -i inventory.ini site.yml — run manually after approving the analysis.

Q: Who made the final decision to apply the playbook?

The student, after reviewing the risky-change report and Claude Code's analysis.

Q: What evidence proves the VM is still reachable after the change?

Running ansible web -i inventory.ini -m ping and receiving epicbook | SUCCESS with pong.

Q: Why should the risk review be run again after applying?

Running the review after gives another view of the system. It confirms what changes remain and checks for unexpected differences after the playbook has been applied.

Q: What could go wrong if an AI agent applied Ansible changes automatically?

It could cause service downtime, change firewall rules, modify permissions, remove files, install incompatible packages, or affect the server in unexpected ways. Keeping a human in the approval loop reduces all of these risks.

Next-Time Quick Checklist

Follow this order on your next Ansible risk-review assignment.

1	Verify Ansible can reach the VM: ansible web -m ping
2	Check playbook syntax: ansible-playbook --syntax-check
3	Create CLAUDE.md with explicit safety rules at project root
4	Define what Claude Code is allowed to do (Bash, Read, Grep only)
5	Write the Bash risk-review script with extract_changed_tasks()
6	Define the four high-risk categories in the script
7	Set exit codes: 0 = healthy, 1 = warning, 2 = risky
8	Run the baseline: ansible-playbook --check --diff
9	Check the Bash report — note exit code and changed tasks
10	Ask Claude Code to analyze the report (read-only)
11	Confirm Claude Code did not edit any files
12	To test risk detection: add a removal/permission task to the playbook
13	Re-run the Bash script and check exit code changes to 2
14	Verify Claude Code flags the risky task in its analysis
15	Review the full analysis yourself before proceeding
16	Run the real playbook manually only after approving
17	Verify the VM is still reachable after the change
18	Run the risk review again to confirm post-apply state

If something is unclear, check these first:

	SYMPTOM	LIKELY CAUSE
1	**Exit code stays 0 even with risky tasks**	Risk category keywords not matched — check your classify logic
2	**Claude Code tries to edit files**	CLAUDE.md safety rules not set or skill permissions too broad
3	**Ansible ping fails before review**	EC2 IP changed or SSH key issue — fix inventory first
4	**--check runs but --diff shows nothing**	Module does not support diff mode — still valid, just no diff output
5	**Script runs but report is empty**	Output capture variable may be empty — check script's capture logic

The Big Lessons from This Assignment

If you forget everything else, keep these.

--check --diff = Dry Run	Preview what Ansible would change without touching the server.
Bash = Deterministic Work	Runs the playbook, captures output, classifies risk, returns exit code.
Claude Code = Analysis Only	Reads the Bash report and explains findings. Never applies changes.
CLAUDE.md = Safety Rules	Defines what Claude Code can and cannot do in this project.
Exit Code 2 = Stop and Review	Risky change detected. A human must approve before real apply.
Human Approval = Non-negotiable	Every real infrastructure change requires a human decision.
Post-Apply Review = Good Practice	Run the risk review again after applying to confirm the final state.
Agentic Loop = Gather + Analyze + Approve	The three-phase pattern keeps humans in control of automation.

This guide was built from a real assignment. Every lesson came from actually doing the work — including the mistakes. If it helped you, share it and pass it on to someone who is just getting started.